

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY4. 딥러닝 네트워크 모델 설계

목 차

1. 딥러닝 네트워크 모델 설계		3
2. 출력층 설계와 손실함수		7
3. 딥러닝 모델 종류		17

1. 딥러닝 네트워크 모델 설계

딥러닝 네트워크 모델 설계는 단순히 층(Layer)을 여러 개 쌓는 작업이 아니라, 해결하고자 하는 문제를 분석하고 데이터의 특성을 고려하여 최적의 신경망 구조를 구성하는 과정이다.

모델 설계 단계에서는 입력 데이터의 형태를 분석하고, 적절한 계층 구조를 선택하며, 활성화 함수와 손실 함수, 최적화 알고리즘 등을 결정해야 한다.

먼저 딥러닝 모델 설계의 첫 단계는 문제 정의이다.

해결하려는 문제가 분류(Classification)인지, 회귀(Regression)인지, 객체 탐지(Object Detection)인지, 자연어 처리(NLP)인지, 이미지 생성인지에 따라 사용하는 네트워크 구조가 달라진다.

예를 들어 고양이와 개를 구분하는 문제라면 분류 모델을 설계해야 하며, 집값을 예측하는 문제라면 회귀 모델을 설계해야 한다.

다음 단계는 입력 데이터 분석이다.

입력 데이터의 특성에 따라 적절한 신경망 구조를 선택해야 한다.

숫자 데이터나 표 형식 데이터(Tabular Data)는 다층 퍼셉트론(Multi Layer Perceptron, MLP)을 주로 사용한다.

이미지 데이터는 공간적 특징을 추출할 수 있는 합성곱 신경망(CNN)을 사용하며, 문장이나 시계열 데이터는 순환 신경망(RNN), LSTM, GRU 또는 Transformer 구조를 사용한다.

입력층(Input Layer)은 데이터가 처음 들어오는 층이다.

입력층의 노드 수는 데이터의 특성 수(Feature)의 개수와 동일하게 설정한다.

예를 들어 학생의 성적 데이터가 국어, 영어, 수학 세 개의 특성으로 구성되어 있다면 입력층의 노드 수는 3개가 된다.

그 다음에는 은닉층(Hidden Layer)을 설계한다.

은닉층은 입력 데이터에서 중요한 특징을 추출하는 역할을 수행한다.

은닉층의 개수와 노드 수는 문제의 복잡도에 따라 결정된다.

일반적으로 단순한 문제는 1~2개의 은닉층으로 충분하지만, 복잡한 이미지나 자연어 문제는 수십 개 또는 수백 개의 층을 사용할 수 있다.

은닉층의 각 뉴런은 입력값과 가중치를 곱한 후 편향(Bias)을 더하여 출력값을 계산한다.

$$z = \sum_{i=1}^n w_i x_i + b$$

여기서 (x_i) 는 입력값, (w_i) 는 가중치, (b) 는 편향을 의미한다.

계산된 값은 활성화 함수(Activation Function)를 통과하여 다음 층으로 전달된다.

활성화 함수는 신경망에 비선형성을 부여하여 복잡한 패턴을 학습할 수 있도록 한다.

대표적인 활성화 함수로는 Sigmoid, Tanh, ReLU, Leaky ReLU, ELU, Swish 등이 있다.

현재 대부분의 딥러닝 모델에서는 계산이 간단하고 기울기 소실 문제가 적은 ReLU 함수를 주로 사용한다.

ReLU 함수는 다음과 같이 표현된다.

$$f(x) = \max(0, x)$$

은닉층을 거친 데이터는 출력층(Output Layer)으로 전달된다.

출력층의 구조는 문제 유형에 따라 달라진다.

이진 분류(Binary Classification)의 경우 출력 노드가 1개이며 Sigmoid 함수를 사용한다.

다중 분류(Multi-Class Classification)의 경우 클래스 수만큼 출력 노드를 사용하고 Softmax 함수를 적용한다.

회귀 문제에서는 활성화 함수를 사용하지 않거나 선형 함수를 사용한다.

다중 분류 문제에서 Softmax 함수는 각 클래스에 속할 확률을 계산한다.

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

모델의 예측 결과와 실제 정답 사이의 차이를 계산하기 위해 손실 함수(Loss Function)를 사용한다.

손실 함수는 모델이 얼마나 잘못 예측했는지를 수치로 나타낸다.

이진 분류에서는 Binary Cross Entropy, 다중 분류에서는 Categorical Cross Entropy, 회귀에서는 MSE(Mean Squared Error)를 주로 사용한다.

평균제곱오차(MSE)는 다음과 같이 계산된다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

손실 함수가 계산되면 역전파(Backpropagation)를 수행하여 오차를 각 층으로 전달한다.

역전파는 출력층에서 입력층 방향으로 진행되며, 경사하강법(Gradient Descent)을 이용하여 가중치를 수정한다.

가중치 갱신은 다음 식으로 표현된다.

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

여기서 (η)는 학습률(Learning Rate)이며, 너무 크면 학습이 불안정해지고 너무 작으면 학습 속도가 느려진다.

최적화 알고리즘은 손실 함수를 최소화하기 위해 사용된다.

대표적으로 SGD, Momentum, RMSProp, Adam이 있으며, 현재 가장 널리 사용되는 방법은 Adam이다.

Adam은 Momentum과 RMSProp의 장점을 결합하여 빠르고 안정적인 학습을 수행한다.

모델 설계 시에는 과적합(Overfitting)을 방지하는 기법도 함께 고려해야 한다.

대표적인 방법으로 Dropout, Batch Normalization, Early Stopping, Data Augmentation이 있다.

Dropout은 학습 과정에서 일부 뉴런을 무작위로 제거하여 특정 뉴런에 과도하게 의존하는 현상을 방지한다.

최종적으로 딥러닝 네트워크 모델 설계 과정은 다음과 같은 순서로 진행된다.

1. 문제 정의
2. 데이터 수집 및 분석
3. 입력층 설계
4. 은닉층 개수 및 노드 수 결정
5. 활성화 함수 선택
6. 출력층 설계
7. 손실 함수 선택
8. 최적화 알고리즘 선택
9. 학습률 및 하이퍼파라미터 설정
10. 과적합 방지 기법 적용
11. 모델 학습 및 평가
12. 성능 개선 및 재설계

즉, 딥러닝 네트워크 모델 설계란 입력 데이터가 입력층에서 시작하여 은닉층을 통해 특징을 학습하고, 출력층에서 결과를 생성한 뒤, 손실 함수와 역전파를 이용해 가중치를 반복적으로 수정하면서 최적의 예측 모델을 만드는 전체 과정을 의미한다. 이는 인간의 뇌 신경망 구조를 모방하여 데이터 속의 복잡한 패턴과 관계를 학습하는 핵심 기술이다.

2. 출력층 설계와 손실함수

출력층 설계는 딥러닝 모델이 최종적으로 어떤 형태의 답을 내보낼지 결정하는 과정이다. 손실함수는 그 답이 정답과 얼마나 다른지 계산하는 기준이다. 두 개념은 따로 생각하면 안 되고, 항상 함께 설계해야 한다.

1. 출력층 설계의 의미

딥러닝 모델은 입력층, 은닉층, 출력층으로 구성된다. 입력층은 데이터를 받아들이고, 은닉층은 데이터 속 특징을 학습하며, 출력층은 최종 예측 결과를 만든다.

예를 들어 어떤 모델이 사진을 보고 동물의 종류를 맞힌다고 하면, 은닉층은 사진 속의 선, 색, 모양, 눈, 귀, 털 같은 특징을 추출한다. 출력층은 그 특징을 바탕으로 "고양이인지, 강아지인지, 토끼인지"를 최종적으로 판단한다.

즉, 출력층은 모델의 마지막 판단 단계이다.

2. 문제 유형에 따른 출력층 설계

출력층은 문제 유형에 따라 다르게 설계한다.

회귀 문제

회귀는 숫자를 예측하는 문제이다. 예를 들어 집값 예측, 온도 예측, 매출 예측, 주가 예측 등이 있다.

예측해야 하는 값이 하나라면 출력 노드는 1 개로 설계한다.

출력층 노드 수 = 1

예를 들어 집값을 하나의 숫자로 예측한다면 출력층은 다음과 같은 형태가 된다.

입력 데이터 → 은닉층 → 출력값 1 개

이때 출력값은 확률이 아니라 실제 숫자이다. 따라서 출력층에 보통 특별한 활성화 함수를 사용하지 않는다. 이를 선형 출력이라고 한다.

이진 분류 문제

이진 분류는 두 가지 중 하나를 고르는 문제이다. 예를 들어 합격/불합격, 정상/불량, 스팸/정상 메일, 질병 있음/없음 같은 문제가 있다.

이진 분류에서는 출력 노드를 보통 1 개로 설계한다.

출력층 노드 수 = 1

출력값은 하나의 점수로 나오고, 이 점수를 확률처럼 해석하기 위해 Sigmoid 함수를 사용할 수 있다.

Sigmoid 함수는 어떤 값을 0 과 1 사이의 값으로 바꾼다.

0 에 가까우면 0 번 클래스

1 에 가까우면 1 번 클래스

예를 들어 출력값이 0.85 라면 모델은 "1 번 클래스일 가능성이 높다"고 판단한다.

다중 분류 문제

다중 분류는 여러 개의 클래스 중 하나를 고르는 문제이다. 예를 들어 손글씨 숫자 0~9 분류, 동물 5 종 분류, 의류 10 종 분류 등이 있다.

이 경우 출력층의 노드 수는 클래스 개수와 같아야 한다.

출력층 노드 수 = 클래스 개수

예를 들어 10 개 클래스를 분류한다면 출력층 노드는 10 개가 된다.

출력값 = [클래스 0 점수, 클래스 1 점수, 클래스 2 점수, ..., 클래스 9 점수]

모델은 이 점수들 중 가장 큰 값을 가진 클래스를 최종 예측값으로 선택한다.

다중 분류에서는 Softmax 함수를 사용하여 각 클래스에 속할 확률을 계산할 수 있다.

각 클래스 점수 → Softmax → 각 클래스 확률

Softmax 를 적용하면 모든 클래스 확률의 합은 1 이 된다.

3. 출력층 활성화 함수 선택

출력층에서 어떤 활성화 함수를 사용할지는 문제 유형에 따라 달라진다.

회귀 문제에서는 보통 활성화 함수를 사용하지 않는다. 숫자 범위가 제한되지 않아야 하기 때문이다.

이진 분류 문제에서는 Sigmoid 를 사용한다. Sigmoid 는 출력값을 0 과 1 사이로 바꾸기 때문에 확률처럼 해석할 수 있다.

다중 분류 문제에서는 Softmax 를 사용한다. Softmax 는 여러 클래스 점수를 확률 분포로 바꾸어 준다.

문제 유형	출력 노드 수	출력층 활성화 함수	의미
회귀	예측값 개수	없음 또는 선형	숫자 예측
이진 분류	1 개	Sigmoid	0 또는 1 분류
다중 분류	클래스 개수	Softmax	여러 클래스 중 하나 선택
다중 라벨 분류	클래스 개수	Sigmoid	여러 항목 동시 선택

다중 라벨 분류는 다중 분류와 다르다. 다중 분류는 여러 개 중 하나만 고르는 문제이고, 다중 라벨 분류는 여러 개가 동시에 정답이 될 수 있는 문제이다. 예를 들어 한 사진에 "사람", "자동차", "강아지"가 동시에 있을 수 있다면 다중 라벨 분류이다.

손실함수(Loss Function)는 딥러닝 모델이 예측한 값과 실제 정답이 얼마나 다른지를 계산하는 함수이다. 모델은 학습 과정에서 손실함수의 값을 최소로 만드는 방향으로 가중치와 편향을 수정한다. 즉, 손실함수는 모델 학습의 기준이 되는 핵심 함수이다.

4. 손실함수의 기본 개념

딥러닝 모델은 입력 데이터를 받아 예측값을 만든다.

입력 데이터 → 모델 → 예측값

예측값이 실제 정답과 같으면 손실값은 작아지고, 예측값이 정답과 많이 다르면 손실값은 커진다.

일반적으로 손실함수는 다음과 같은 형태로 이해할 수 있다.

손실값 = 실제값과 예측값의 차이

수식에서는 실제값을 보통 (y), 예측값을 ($\hat{y}$) 로 표현한다.

y = 실제 정답

$\hat{y}$ = 모델의 예측값

L = 손실값

5. 평균제곱오차 MSE

MSE 는 Mean Squared Error 의 약자이며, 평균제곱오차라고 한다. 회귀 문제에서 가장 많이 사용되는 손실함수이다.

공식은 다음과 같다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

여기서 (n)은 데이터 개수, (y_i)는 실제값, ($\hat{y}_i$)는 예측값이다.

MSE 는 실제값과 예측값의 차이를 제곱한 뒤 평균을 낸다. 차이를 제곱하기 때문에 오차가 클수록 손실값이 급격히 커진다.

예를 들어 실제값이 100 이고 예측값이 90 이면 오차는 10 이다.

$$(100 - 90)^2 = 100$$

실제값이 100 이고 예측값이 50 이면 오차는 50 이다.

$$(100 - 50)^2 = 2500$$

즉, MSE 는 큰 오차에 매우 강한 벌점을 준다.

MSE 는 집값 예측, 온도 예측, 매출 예측처럼 연속적인 숫자를 예측하는 문제에 적합하다.

```
criterion = nn.MSELoss()
```

6 평균절대오차 MAE

MAE 는 Mean Absolute Error 의 약자이며, 평균절대오차라고 한다.

공식은 다음과 같다.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE 는 실제값과 예측값의 차이에 절댓값을 씌운 뒤 평균을 낸다.

예를 들어 실제값이 100 이고 예측값이 90 이면 다음과 같다.

$$|100 - 90| = 10$$

실제값이 100 이고 예측값이 50 이면 다음과 같다.

$$|100 - 50| = 50$$

MAE 는 오차를 제공하지 않기 때문에 MSE 보다 이상치에 덜 민감하다. 데이터에 극단적으로 큰 값이나 작은 값이 있을 때 MSE 보다 안정적으로 사용할 수 있다.

```
criterion = nn.L1Loss()
```

PyTorch에서는 MAE 를 nn.L1Loss()로 사용한다.

7. Huber Loss

Huber Loss 는 MSE 와 MAE 의 장점을 결합한 손실함수이다.

공식은 다음과 같다.

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2, & |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta), & |a| > \delta \end{cases}$$

여기서 $a = y - \hat{y}$ 이다.

오차가 작을 때는 MSE 처럼 제곱 오차를 사용하고, 오차가 클 때는 MAE 처럼 절댓값 오차를 사용한다.

즉, 작은 오차는 민감하게 학습하고, 너무 큰 이상치에는 과하게 흔들리지 않도록 만든다.

Huber Loss 는 회귀 문제에서 이상치가 섞여 있을 때 유용하다.

`criterion = nn.HuberLoss()`

8. 이진 교차 엔트로피 BCE

BCE 는 Binary Cross Entropy 의 약자이다. 이진 분류 문제에서 사용된다.

이진 분류는 정답이 0 또는 1 인 문제이다.

예시는 다음과 같다.

스팸 메일 여부: 스팸 / 정상

질병 여부: 있음 / 없음

불량 여부: 불량 / 정상

합격 여부: 합격 / 불합격

공식은 다음과 같다.

$$BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

여기서 (y_i)는 실제 정답이고, $\hat{y}_i$ 는 모델이 예측한 확률이다.

예를 들어 실제 정답이 1 인데 모델이 0.95 라고 예측하면 정답에 가까우므로 손실이 작다.

반대로 실제 정답이 1 인데 모델이 0.05 라고 예측하면 정답과 멀기 때문에 손실이 매우 크다.

BCE 는 단순히 맞고 틀림만 보는 것이 아니라, 모델이 얼마나 확신 있게 맞혔는지도 평가한다.

PyTorch 에서는 보통 다음을 더 권장한다.

```
criterion = nn.BCEWithLogitsLoss()
```

BCEWithLogitsLoss 는 Sigmoid 와 BCE 를 함께 계산한다. 따라서 출력층에 Sigmoid 를 따로 넣지 않아도 된다.

9. 다중 클래스 교차 엔트로피 Cross Entropy

Cross Entropy Loss 는 다중 분류 문제에서 가장 많이 사용되는 손실함수이다.

다중 분류는 여러 개의 클래스 중 하나를 선택하는 문제이다.

예시는 다음과 같다.

손글씨 숫자 0~9 분류

동물 이미지 5 종 분류

의류 이미지 10 종 분류

뉴스 기사 카테고리 분류

공식은 다음과 같다.

$$CE = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

여기서 (C)는 클래스 개수이다.

정답이 One-Hot Encoding 형태라면 다음과 같이 표현할 수 있다.

정답 클래스가 2 번인 경우

[0, 0, 1, 0, 0]

이때 정답 위치만 1 이므로 실제 계산에서는 정답 클래스의 예측 확률만 중요하게 사용된다.

예를 들어 정답이 고양이이고 모델이 다음과 같이 예측했다고 하자.

고양이: 0.90

강아지: 0.07

토끼: 0.03

정답 클래스인 고양이에 높은 확률을 주었으므로 손실이 작다.

반대로 다음과 같다면 손실이 크다.

고양이: 0.05

강아지: 0.80

토끼: 0.15

정답 클래스인 고양이에 낮은 확률을 주었기 때문이다.

PyTorch에서는 다음과 같이 사용한다.

```
criterion = nn.CrossEntropyLoss()
```

주의할 점은 PyTorch 의 CrossEntropyLoss 는 내부적으로 Softmax 를 포함한다는 점이다.

따라서 모델 출력층에 Softmax 를 직접 넣지 않는 것이 일반적이다.

10. 다중 라벨 분류 손실함수

다중 라벨 분류는 여러 개의 정답이 동시에 존재할 수 있는 문제이다.

예를 들어 한 사진에 사람, 자동차, 강아지가 동시에 있을 수 있다.

사람: 있음

자동차: 있음

강아지: 있음

고양이: 없음

이 경우 다중 분류처럼 하나만 고르는 것이 아니라, 각 클래스마다 0 또는 1 을 따로 판단해야 한다.

따라서 각 클래스에 대해 이진 분류를 수행하므로 BCEWithLogitsLoss 를 사용한다.

공식은 BCE 와 같다.

$$BCE = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

```
criterion = nn.BCEWithLogitsLoss()
```

11. Negative Log Likelihood Loss

NLLLoss 는 Negative Log Likelihood Loss 의 약자이다.

공식은 다음과 같다.

$$NLL = -\log(P(y))$$

여기서 (P(y))는 정답 클래스에 대한 예측 확률이다.

정답 클래스의 확률이 높으면 손실이 작고, 정답 클래스의 확률이 낮으면 손실이 커진다.

PyTorch에서는 LogSoftmax 와 함께 사용한다.

```
criterion = nn.NLLLoss()
```

일반적으로는 LogSoftmax + NLLLoss 대신 CrossEntropyLoss 를 더 많이 사용한다.

왜냐하면 CrossEntropyLoss 가 내부적으로 이 두 과정을 함께 처리하기 때문이다.

12. KL Divergence Loss

KL Divergence 는 두 확률분포가 얼마나 다른지를 측정하는 손실함수이다.

공식은 다음과 같다.

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

여기서 (P)는 실제 확률분포, (Q)는 모델이 예측한 확률분포이다.

KL Divergence 는 다음과 같은 경우에 사용된다.

지식 증류

확률분포 비교

생성 모델

Variational AutoEncoder

두 분포가 비슷하면 값이 작고, 두 분포가 다르면 값이 커진다.

`criterion = nn.KLDivLoss()`

13. 손실함수 선택 기준

문제 유형에 따라 손실함수를 다르게 선택해야 한다.

문제 유형	출력 형태	대표 손실함수
회귀	연속 숫자	MSELoss, L1Loss, HuberLoss
이진 분류	0 또는 1	BCEWithLogitsLoss
다중 분류	여러 클래스 중 하나	CrossEntropyLoss
다중 라벨 분류	여러 클래스 동시 선택	BCEWithLogitsLoss
확률분포 비교	분포와 분포 비교	KLDivLoss
로그 확률 기반 분류	로그 확률	NLLLoss

3. 딥러닝 모델 종류

딥러닝(Deep Learning)은 인간의 뇌 신경망을 모방한 인공신경망(Artificial Neural Network)을 여러 층으로 깊게 쌓아 복잡한 패턴을 학습하는 기술이다. 딥러닝 모델은 데이터의 종류와 목적에 따라 다양한 구조로 발전해 왔다. 각 모델은 해결하려는 문제에 따라 서로 다른 특징과 장점을 가진다.

1. 퍼셉트론 (Perceptron)

퍼셉트론은 가장 기본적인 인공신경망 모델이다. 1957 년 Frank Rosenblatt 가 제안하였다.

구조는 매우 단순하다.

입력층 → 출력층

예를 들면 다음과 같다.

공부시간 → 합격여부

퍼셉트론은 입력값과 가중치를 곱한 후 합산하여 결과를 출력한다.

$$y=f\left(\sum_{i=1}^n w_i x_i+b\right)$$

장점

- 구조가 단순함
- 계산이 빠름

단점

- XOR 문제 해결 불가능
- 복잡한 패턴 학습 불가능

2. 다층 퍼셉트론 (MLP)

퍼셉트론의 한계를 극복하기 위해 은닉층(Hidden Layer)을 추가한 모델이다.

구조

입력층 → 은닉층 → 출력층

예시

784 → 128 → 64 → 10

MLP 는 이미지 분류, 숫자 예측, 금융 예측 등에 사용된다.

특징

- 역전파(Backpropagation) 사용
- 비선형 문제 해결 가능
- 딥러닝의 기본 구조

3. 심층신경망 (DNN)

DNN(Deep Neural Network)은 MLP 의 은닉층을 여러 개 추가한 구조이다.

구조

입력층

↓

은닉층 1

↓

은닉층 2

↓

은닉층 3

↓

출력층

특징

- 복잡한 패턴 학습 가능
- 높은 표현력
- 대량 데이터 필요

활용

- 음성인식
- 추천시스템
- 금융 예측

4. 합성곱 신경망 (CNN)

CNN(Convolutional Neural Network)은 이미지 처리에 특화된 모델이다.

대표적으로

- 얼굴 인식
- 의료 영상 분석
- 자율주행

등에 사용된다.

구조

입력

↓

Convolution

↓

ReLU

↓

Pooling

↓

Fully Connected

↓

출력

CNN 의 핵심은 Convolution 연산이다.

예시 필터

1 0 -1

1 0 -1

1 0 -1

이미지의 특징을 추출한다.

대표 모델

- LeNet
- AlexNet
- VGG16
- GoogLeNet
- ResNet

5. 순환 신경망 (RNN)

RNN(Recurrent Neural Network)은 순서 정보가 중요한 데이터를 처리하기 위해 개발되었다.

구조

현재 입력

↓

RNN Cell

↑

이전 상태

특징

- 이전 정보를 기억
- 시계열 데이터 처리

활용

- 주가 예측
- 날씨 예측
- 문장 생성

문제점

- 장기 기억 어려움
- Gradient Vanishing 발생

6. LSTM

LSTM(Long Short-Term Memory)은 RNN 의 문제를 개선한 모델이다.

구조

Forget Gate

Input Gate

Output Gate

Cell State

특징

- 장기 기억 가능
- 시계열 데이터에 강함

활용

- 번역
- 음성인식
- 주식 예측

7. GRU

GRU(Gated Recurrent Unit)는 LSTM 을 단순화한 모델이다.

특징

- 구조 단순
- 학습 속도 빠름
- 성능은 LSTM 과 유사

활용

- 자연어 처리
- 시계열 분석

8. AutoEncoder

입력 데이터를 압축했다가 다시 복원하는 모델이다.

구조

입력

↓

Encoder

↓

잠재공간

↓

Decoder

↓

출력

활용

- 차원 축소
- 노이즈 제거
- 이상 탐지

대표 예

- PCA 대체
- 이미지 압축

9. Variational AutoEncoder (VAE)

AutoEncoder 를 발전시킨 생성 모델이다.

특징

- 새로운 데이터 생성 가능
- 잠재공간 학습

활용

- 얼굴 생성
- 이미지 생성

10. GAN

GAN(Generative Adversarial Network)은 생성자와 판별자가 경쟁하며 학습하는 모델이다.

구조

Noise

↓

Generator

↓

가짜 이미지

↓

Discriminator

↓

진짜/가짜 판별

생성자

가짜를 진짜처럼 만듦

판별자

진짜와 가짜를 구별

대표 GAN

- DCGAN
- CycleGAN
- StyleGAN

활용

- 얼굴 생성
- 이미지 복원
- AI 그림 생성

11. Transformer

현재 AI 발전의 핵심 모델이다.

RNN 없이 Attention 만 사용한다.

구조

입력

↓

Self-Attention

↓

Feed Forward

↓

출력

핵심 개념

Attention

중요한 단어에 집중

예

나는 사과를 먹었다.

"먹었다"를 이해할 때 "사과"에 집중한다.

장점

- 병렬 처리 가능
- 긴 문장 처리 가능

12. BERT

BERT 는 Transformer Encoder 기반 모델이다.

특징

- 양방향 문맥 이해
- 문장 이해에 강함

활용

- 문서 분류
- 챗봇
- 검색엔진

13. GPT

GPT 는 Transformer Decoder 기반 생성형 AI 모델이다.

특징

- 텍스트 생성
- 코드 생성
- 질의응답

활용

- ChatGPT
- 문서 작성
- 번역

14. Vision Transformer (ViT)

Transformer 를 이미지 처리에 적용한 모델이다.

구조

이미지



Patch 분할



Transformer



분류

특징

- CNN 없이 이미지 분석
- 대규모 데이터에서 우수

15. Diffusion Model

현재 이미지 생성 AI 의 주류 기술이다.

대표 모델

- Stable Diffusion
- DALL·E

동작 원리

원본 이미지



노이즈 추가



완전한 잡음



노이즈 제거 학습



새 이미지 생성

특징

- 고품질 이미지 생성
- GAN 보다 안정적

딥러닝 모델 발전 순서

Perceptron

↓

MLP

↓

DNN

↓

CNN / RNN

↓

LSTM / GRU

↓

AutoEncoder

↓

GAN

↓

Transformer

↓

BERT / GPT

↓

Vision Transformer

↓

Diffusion Model

현재 실무에서는 데이터 종류에 따라 주로 다음과 같이 사용된다.

데이터 유형	대표 모델
표(Tabular) 데이터	DNN, MLP
이미지 분류	CNN, ViT
객체 탐지	CNN 기반 YOLO
시계열 데이터	LSTM, GRU
자연어 처리	Transformer, BERT
생성형 AI	GPT
이미지 생성	GAN, Diffusion
차원 축소	AutoEncoder
이상 탐지	AutoEncoder, Transformer

최근 AI 연구와 산업 현장에서는 Transformer 계열(LLM)과 Diffusion 계열(이미지 생성)이 가장 활발하게 활용되고 있으며, CNN 과 LSTM 은 특정 분야에서 여전히 중요한 기반 모델로 사용되고 있다.